

[2024 Fall] UNIST CSE221 Data Structures

Programming Assignment #5:

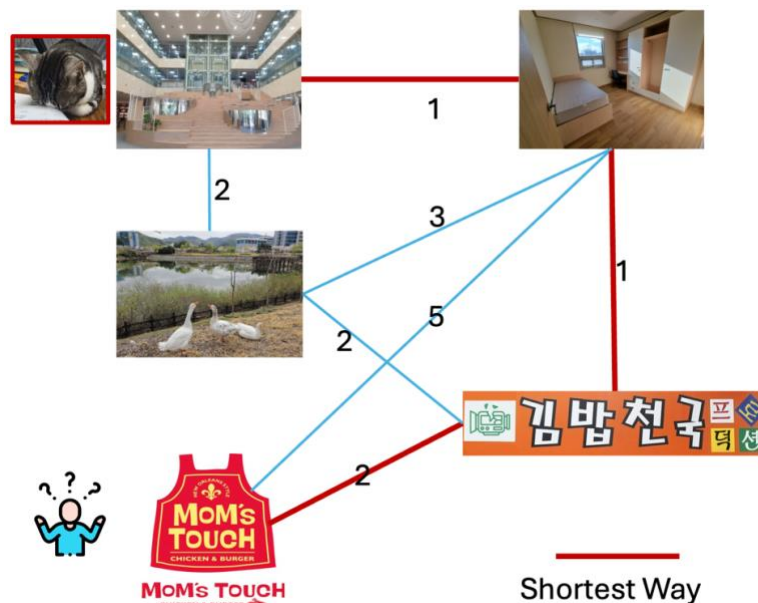
Shortest Way to a Cat

Prof: Taesik Gong (taesik.gong@unist.ac.kr)

TA: Yeon An (anyeon@unist.ac.kr)

Introduction

In this programming assignment, you will develop a program to find the shortest way to reach a cat located somewhere on the UNIST campus using C++. The campus can be modeled as a graph of places connected by paths with distances. This assignment focuses on using different types of data structures to store, retrieve, and process information, specifically using **Graphs** and **Heaps**. You will implement three tasks, each responsible for a specific part of the system. By integrating these data structures and algorithms, you'll gain a deeper understanding of graph representations, priority queues, and shortest path algorithms, as well as practical applications of **Dijkstra's algorithm**.



Assumptions:

- Place names are unique strings (e.g., "Library", "Cafeteria").
- Places are connected with roads specified with distances.

- All distances are positive integers.
- At most one road exists for every pair of places.
- All places are connected and form an undirected graph with no self-loops.

Part 1: Graph Representations Using Adjacency List

Implement a graph to represent the campus, where places are vertices and distances are weights in edges. Use an adjacency list we learned in the class to efficiently represent the graph.

Please refer to the skeleton code for the methods you need to implement and their detailed specifications.

Part 2: Priority Queue with Heap

Implement a priority queue using a (binary) heap. Consider methods that we learned in the class. Use the “vector-based heap implementation” we learned in the class (9. Priority Queues-Heap). This priority queue will be used to select the next place to visit based on the shortest known distance in Dijkstra’s algorithm.

Please refer to the skeleton code for the methods you need to implement and their detailed specifications.

Part 3: Find the Shortest Path to Reach a Cat

Using Dijkstra’s algorithm, find the shortest path to the cat’s location on campus.

Please refer to the skeleton code for the methods you need to implement and their detailed specifications.

Hint 1: Use the Graph from Part 1 for graph representation.

Hint2: Use the Heap from Part 2 as the priority queue in Dijkstra’s algorithm.

Skeleton Code

We provide the following files for you to start with:

- `main.cpp`

- graph.h
- graph.cpp
- heap.h
- heap.cpp
- dijkstra.h
- dijkstra.cpp
- Makefile

Your job is to complete the implementation of all parts. You can change the skeleton code except for specified areas (e.g., some areas *.h files). We provide an example **Makefile for compilation. You can add your own files if necessary, but note that you should explain them in ReadMe.txt and address them in your Makefile. You can test your code with main.cpp we provide, but the code written in main.cpp does not cover all the test cases in grading.**

Submission

- **Submission Format:** Compress all the relevant files (*.cpp, *.h, Makefile, ReadMe.txt, etc.) into a single **zip** file named **STUDENT_ID.zip** (e.g., 20242222.zip) and submit it.
 - a. all necessary source files (*.cpp and *.h)
 - b. **Makefile:** Create your own Makefile to compile your project. You can use the Makefile we provided. This will be part of your grade, so make sure it compiles your project correctly when typing make in the terminal. See the “compilation” section in the grading criteria below for details.
 - c. **ReadMe.txt:** Include a Readme file explaining any specific considerations for grading your project or any deviations from the instructions.
- **Deadline: 13th December 11:59PM**
 - a. **Late Submission:** The late submission follows the policy that was explained in the course overview lecture (see lecture notes in Blackboard).

Restrictions

- **Allowed Libraries:** You are permitted to use **only** the following C++ standard libraries: `<iostream>`, `<string>`, `<exception>`, `<stdexcept>`, `<cstdlib>`.
- **C++ Version:** It is recommended that you use a recent version of C++ for this assignment. **Minimum Version: C++11. Recommended Version: C++17 or higher.**

Example Output

Below is the console output of running main.cpp with correct implementation.

```
Testing Graph Implementation:
Dorm MomsTouch 10
Dorm KimbapHeaven 3
KimbapHeaven MomsTouch 5

Is MomsTouch adjacent to KimbapHeaven? Yes

Graph after erasing edge between MomsTouch and KimbapHeaven:
Dorm MomsTouch 10
Dorm KimbapHeaven 3

Testing Heap Implementation:
4
5 10
20 15

Heap after removing min:
5
15 10
20

Heap after replacing key at index 2 with 3:
3
5 10
20

Testing Dijkstra's algorithm:
Fastest path from MomsTouch to Library:
MomsTouch KimbapHeaven Dorm Library
Fastest distance from MomsTouch to Library:
4
```

Grading Criteria

- **Submission format**
 - It is mandatory to ensure the integrity of the provided header files. Your submission should follow the instructions in the “submission” section.
- **Compilation**
 - The “**make**” command MUST work to create the “**main**” program (see the example Makefile provided).
 - While you only need to submit part of the source files, assume that the provided *.h files and main.cpp will be included during compilation, so you don’t need to worry about compilation errors due to missing those files.
 - If compilation fails, the score will be zero.

- If a runtime exception halts the program (e.g., a crash due to a segmentation fault), the score will be based on progress up to the point of termination.

- **Main Parts**

- Various test cases will be used to assess the functionality of each requirement described in this document. Scores will only be given to correctly passed test cases.